Zig — A Working Book

A visual, example-driven tour of a small language with big ideas

Jonghwan Lee

2026

Zig — A Working Book

Contents

Zig — A Working Book

Zig — A Working Book

CHAPTER

Preface

This book is about a small, careful programming language called Zig, and about the ideas that make it worth learning. Zig is not the first systems language to come out of the post-C world — Rust, Nim, Odin, Jai, and a dozen others have all staked out different corners of the same problem — but it is the one that keeps surprising me. It is, in its good moments, simultaneously the lowest-level language I have enjoyed writing in and the highest-leverage one.

I wrote it because I had to. I work on systems that care about latency and memory, I like knowing exactly what my code compiles into, and I have a stubborn allergy to hidden control flow. Zig's promise — no hidden allocation, no hidden control flow, no operator overloading, no exceptions, and a compile-time language that is *just your language, run early* — kept answering objections I had not known I had. I wanted a book that treated those ideas as the centre of gravity, not as a footnote to the usual "here is a for-loop" chapter. So I wrote one.

Who this book is for

You should be comfortable in at least one other language at a similar level — C, C++, Rust, Go, Nim, or the systems-y subset of Swift will all do. I will not explain what a pointer is; I will explain what Zig means by the four different pointer types and why you should care. If you have only written Python or JavaScript, this book will still make sense, but Chapter 3 (and some of Chapter 5) will be more work. That is fair: systems programming is more work, and Zig does not pretend otherwise.

How to read it

The book is six parts.

Part I is foundations — why Zig, how to install it, and the subset of the core language you can read in an afternoon. If you are in a hurry and you know one of the languages above, you can read Part I in an hour and skim the bits you do not need.

Part II is values and memory: arrays, slices, pointers, optionals, and — most importantly — allocators. Every other part of the book assumes you understand this one.

Part III is comptime: the compile-time evaluation model that lets Zig do generics, reflection, and code generation in a single, honest mechanism. If you came to Zig from Rust or C++, this is the part where your assumptions about templates and macros get rearranged.

Part IV is error handling, resource management, and testing — three topics that the language treats as deeply connected, and that I think most other languages get wrong in subtly different ways.

Part V is the ecosystem: the build system (which is also Zig), linking and cross-compiling C, and the parts of the standard library you are most likely to reach for.

Part VI is a worked project: a small search tool called `zgrep` that uses something from every earlier chapter, and a short closing chapter on shipping the binaries you have been writing.

The chapters cross-reference each other but stand alone. If you already write Zig and you bought this for the comptime chapter, read the comptime chapter.

Conventions

Code blocks are Zig unless the label says otherwise. A block that I expect you to run looks like this:

```zig
const std = @import("std");

pub fn main() !void {
    std.debug.print("hello, {s}\n", .{"world"});
}
```

Output from `zig run` or `zig build run` follows in a plain block:

```
hello, world
```

Callouts come in three colours:

> **Note**
> A note is a sidebar that is worth reading but that you could skip on the first pass without losing the thread.

> **Tip**
> A tip is a shortcut or a habit that saves a real amount of time once you adopt it.

> **Warning**
> A warning marks a sharp edge — a place where the language will let you do the wrong thing and you will not find out until later.

Zig versions

Every language that is still being designed has a period where code ages badly. Zig is in one of those periods right now. Examples in this book target **Zig 0.14.x**, which was the current release when I wrote the last full pass; blocks that depend on a later feature are labelled with the version they need. If a snippet does not compile for you, run `zig version`, match it against the label on the block, and — if the version is right — write to me.

Acknowledgements

Andrew Kelley and the Zig Software Foundation built the language these pages describe; every good idea in this book is theirs, and every mistake is mine. The Zig community on Ziggit, Discord, and GitHub has been unusually patient with dumb questions from people writing books about the language, and I have been one of those people. Thank you.

A final word before Chapter 1: Zig is smaller than it looks. You can fit the whole language on a single cheat sheet. What takes time is not the syntax but the *taste* — the feel for where to put an arena, when to reach for comptime, how to choose an error set. The chapters that follow are about building that taste.

CHAPTER

Why Zig

Every systems language is trying to answer the same question: *how do we keep the control C gives us without the bugs C gives us?* The two dominant answers so far have been, roughly, "add a careful type system on top" (Rust) and "make garbage collection cheap enough that you stop caring" (Go). Zig's answer is different, and a little contrarian: keep the control, add almost nothing on top, and make the things C leaves implicit — allocation, control flow, generics — so explicit that the whole class of bug stops being reachable.

This chapter is a tour of that idea. By the end of it you will have installed Zig, written and run a small program, and seen — in a single diagram — the four ideas that the rest of the book returns to over and over. We will move quickly; later chapters come back to every piece in detail.

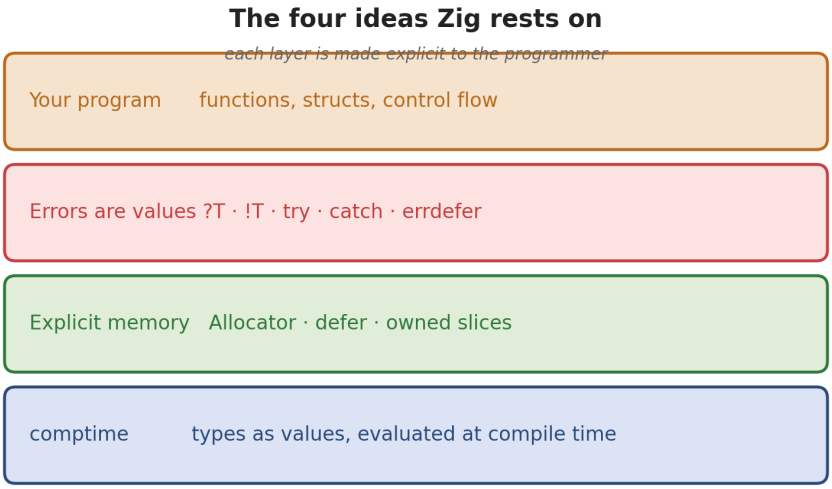
1.1 A language you can fit in your head

The practical claim Zig makes is that you should be able to read any Zig expression and predict what machine code it will compile into. That sounds obvious, but the way most languages grow, it is not. A single `+` in C++ might be integer addition, it might be a function call into `operator+`, it might allocate, it might throw. In Python a `+` might rebind a global. In Go a `+` might trigger a GC write barrier. In Zig, `a + b` is integer addition on two values of the same type, and nothing else. If you want wrapping arithmetic you write `a +% b`. If you want saturating, `a +| b`. If `a` can be null, the compiler refuses the expression until you unwrap. That is the whole design philosophy, enforced consistently.

The other claim — the one that takes longer to believe — is that *less* can genuinely be more. Zig has no macros, no operator overloading, no traits/interfaces/typeclasses, no inheritance, no destructors, no exceptions, no hidden allocations, and no hidden function calls. What it has instead is one good compile-time evaluation mechanism (Chapter 7), one good error-handling mechanism (Chapter 10), and one good memory-management idiom (Chapter 5). Most of the features you miss turn out to be expressible in terms of those three.

> **Note**
> "Fit in your head" is a design aesthetic, not a measurable property. The current Zig compiler implements a language specification of about thirty pages. For comparison, the C11 standard is six hundred, and C++23 is closer to two thousand.

1.2 The four pillars



The four ideas Zig rests on. Each one is made explicit to the programmer; together, they are almost the entire language.

comptime. Zig runs a subset of its own language at compile time. Every type is a value; every value has a type; the same function can be invoked on compile-time constants to produce another compile-time constant. This is how generics work (types are function parameters), how reflection works (`@typeInfo` returns a struct you can iterate over), and how string manipulation at the type level works (building a SQL query string is the same thing as building a SQL query *type*). Chapter 7 is a whole chapter on it.

Explicit memory. There is no garbage collector, and the standard library has no global allocator. A function that can allocate always takes an `Allocator` argument. You pick the concrete allocator exactly once, at the top of your program, and then everything below that

point is polymorphic over the choice. In practice you will spend more of your day thinking about allocation than you would in C or Rust — because it is *visible* — and then, because it is visible, you will run into fewer memory bugs. Chapter 5 is about this.

Errors are values. Zig has no exceptions. A function that can fail declares its error set and returns an "error union" — a tagged value that is either a `T` or an error. Control flow around errors uses two keywords (`try` and `catch`) and no stack unwinding, which means the compiler can lay out your code with the happy path straight through and the error path as a separate, cold branch. Chapter 10 is about this.

No hidden control flow. There are no destructors, no operator overloads, no implicit conversions, no type coercion that costs more than a cast, and no runtime dispatch you did not ask for. If you see a `+`, it is integer addition. If you see a `deinit()` call, something explicitly called it. This sounds austere, but it adds up to a language where, once you can read a function, you can predict its cost.

1.3 A first end-to-end example

The goal of this section is to take you through a small Zig program in less than a page: install the compiler, write a file, run it. We will count words in a string.

Installing Zig

On macOS: `brew install zig`. On Linux, grab a release tarball from `ziglang.org/download` and add it to your `PATH`; there are no distribution packages we trust yet. On Windows, unzip the release archive and add it to your `PATH`. To confirm:

```
zig version
```

```
0.14.0
```

Anything ≥ 0.14 will do for this book. Chapter 2 has the longer version — virtual environments, editor setup, and how to pin the compiler version per project.

Your first program

Make a directory, step into it, and ask Zig to scaffold a project:

```
mkdir hello && cd hello
zig init
```

You get a `build.zig`, a `build.zig.zon`, and a `src/main.zig`. Open `src/main.zig` and replace its contents with:

```
const std = @import("std");

pub fn main() !void {
    const stdout = std.io.getStdOut().writer();

    const sentence = "Zig is a small language with big ideas.";
    var count: usize = 0;
    var it = std.mem.tokenizeScalar(u8, sentence, ' ');
    while (it.next()) |_| count += 1;

    try stdout.print("{d} words\n", .{count});
}
```

Run it:

```
zig build run
```

```
8 words
```

Four things worth noticing, because they come back in every Zig program:

1. `const std = @import("std");` is how you pull in the standard library. `@import` is a compile-time builtin — it returns a `struct` whose fields are the library's declarations. You will see a lot of `@...` builtins. They are the only form of "magic" in the language, and there are about a hundred of them, all listed in the language reference.
2. `pub fn main() !void` — the return type is `!void`. The `!` marks it as an error-union: the function might return an error (say, because writing to stdout failed) or it might return nothing. The caller — in this case, the runtime — handles the error by printing a stack trace and exiting with a non-zero code.
3. `std.mem.tokenizeScalar(u8, sentence, ' ')` — a generic function invoked with a *type* (`u8`) as its first argument. That is normal in Zig, because types are values. Chapter 8 shows you how to write generic code of your own.

4. `try stdout.print(...)` — `try` unwraps an error union and short-circuits with the error if one is present. If `print` returns an error, `main` returns that error. Chapter 10 makes this rigorous.

In about twenty lines, we have touched the standard library, the compile-time import mechanism, generic functions, error handling, and I/O. We have not had to decide which allocator to use, because `tokenizeScalar` does not allocate — it returns an iterator over the original string. We will meet allocation in the next example.

A word-count CLI

Let us turn the one-liner into something closer to a real tool — a `wc -w` that reads a file whose name is given as an argument.

```
const std = @import("std");

pub fn main() !void {
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer _ = gpa.deinit();
    const a = gpa.allocator();

    var args = try std.process.argsWithAllocator(a);
    defer args.deinit();
    _ = args.next(); // program name
    const path = args.next() orelse {
        std.debug.print("usage: wc <file>\n", .{});
        return;
    };

    const bytes = try std.fs.cwd().readFileAlloc(a, path, 1 << 20);
    defer a.free(bytes);

    var count: usize = 0;
    var it = std.mem.tokenizeAny(u8, bytes, " \t\n\r");
    while (it.next()) |_| count += 1;

    const stdout = std.io.getStdOut().writer();
    try stdout.print("{d} {s}\n", .{ count, path });
}
```

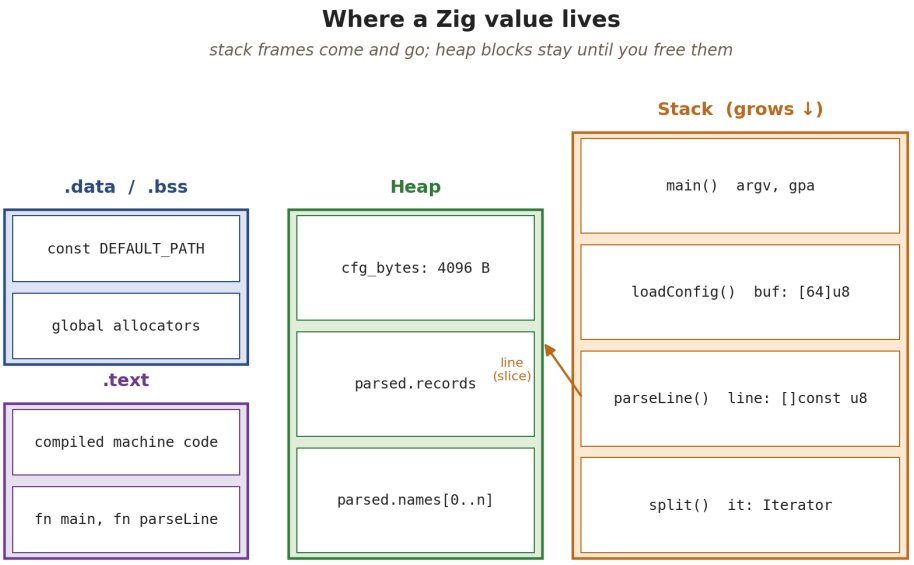
```
$ zig build run -- README.md
42 README.md
```

The shape of this program is the shape every Zig program has. An allocator at the top, `defer` calls that clean up in reverse order, `try` on every call that can fail, a quick exit on missing arguments, and no hidden anything. The only thing that might surprise a reader from a GC language is that we decided how much memory we were willing to read (`1 << 20` bytes, or one megabyte) at the call site. That is not a Zig quirk — it is just that systems programming languages ask you to name the bound explicitly, and Zig refuses to make the choice for you.

> **Warning**
> `GeneralPurposeAllocator` is a *debug-oriented* allocator. It detects leaks, double-frees, and use-after-free, which makes it a good default during development but slower than `page_allocator` or `c_allocator` in release mode. Chapter 5 covers when to reach for which.

1.4 Where values live

Before we move on, it is worth looking at a picture of where the different parts of a Zig program live in memory. Every systems programmer has a mental model of this — a diagram of stack frames and heap blocks — but the Zig version is especially clean, because the language never puts anything anywhere you did not ask it to.



*Where the pieces of a Zig program live. Stack frames grow downward and vanish at the end of each call; heap blocks stay alive until the allocator that owns them releases them; `.data`, `.bss`, and `.text` are baked into the binary. A slice on the stack is a pointer-plus-length that *refers* to bytes somewhere else — typically on the heap.*

Every variable you declare ends up in one of four regions:

- **The stack** holds local variables of the currently-running functions. A stack frame comes into existence when a function is called and disappears when it returns. Values on the stack are free to allocate — the CPU is already doing it. But they cease to exist the moment their function returns, which is why you cannot return a pointer to a local.
- **The heap** holds memory requested at runtime from an allocator. Heap memory stays alive until it is explicitly freed (or, for an arena, until the arena is destroyed). Every allocator in Zig is just an object that implements the `std.mem.Allocator` vtable.
- `.data` / `.bss` hold statically-initialised globals — for example, the bytes of the string literal `"hello\n"`. They are part of the binary; they are there for the whole process lifetime.
- `.text` holds the compiled machine code itself.

The thing to take away from the picture is that a *slice* (`[]u8`) — Zig's usual "range of bytes" type — is a stack value that points into one of the other regions. It does not own what

it points to; it just tells you where it is. Chapter 4 spends twenty pages on this one idea, because slices are the single most common source of confusion when people first come to the language.

1.5 What isn't in Zig

It is worth being honest about what Zig does not do well.

A stable language. Zig is pre-1.0. The core team is happy to change the language when they find a better design — the build system has changed shape three times since I started writing this book, and async/await has been removed and is being redesigned. If you want stability above all else, come back in a few years, or use Rust or Go.

A huge ecosystem. Zig's package registry is young, and most problems you want to solve will require wrapping a C library. The upside is that C interop is probably Zig's strongest feature (Chapter 14); the downside is that you will sometimes be the first person to write a Zig binding for something.

Polished abstractions for very high-level code. There is no Zig web framework that feels like Django, no ORM, no GUI toolkit that competes with Qt or AppKit. You can build these things — people are — but you will feel the lack of them if you try to write a CRUD app in Zig without sourcing the pieces yourself.

Automatic memory management. Zig refuses to hide allocation, which means you will spend time thinking about when to free. If you do not want to, that is a completely reasonable preference, and you will be happier in Go.

None of these are dealbreakers for systems work. They are worth knowing so you know when to step outside the language.

1.6 How this book is organized

The book is six parts.

Part I (Chapters 1–3) is foundations: this tour chapter, a setup chapter, and a concentrated crash course in the parts of the core language you actually need.

Part II (Chapters 4–6) is values and memory, top to bottom: arrays and slices, allocators, and the aggregate types (structs, enums, unions).

Part III (Chapters 7–9) is comptime — generics, reflection, and code generation — which is where Zig is most different from every language you have used before.

Part IV (Chapters 10–12) is control flow and correctness: errors, `defer`, testing.

Part V (Chapters 13–15) is the ecosystem — the build system, C interop, concurrency.

Part VI (Chapters 16–17) is a worked project and a closing chapter on shipping.

You can read linearly, or you can skip to whichever part answers a question you have right now. The chapters cross-reference each other, but each one is written to stand alone.

1.7 Setup sanity check

Before you move on, make sure the following runs without error. It's the smallest possible version of the program we just walked through:

```
const std = @import("std");

pub fn main() !void {
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer _ = gpa.deinit();
    const a = gpa.allocator();

    const xs = try a.alloc(u32, 5);
    defer a.free(xs);
    for (xs, 0..) |*x, i| x.* = @intCast(i * i);

    std.debug.print("{any}\n", .{xs});
}
```

If it prints `{ 0, 1, 4, 9, 16 }` and exits cleanly (no leak report from the GPA), every piece of the toolchain the book uses is working. If anything fails, jump to Chapter 2 — it is about getting your environment into a shape where the rest of this book will work the first time.

1.8 Exercises

1. Modify the word-count program to count *lines* as well as words, printing both numbers. Hint: `std.mem.splitScalar(u8, bytes, '\n')` returns an iterator over lines.

2. Change the maximum file size in `readFileAlloc` from 1 MiB to the actual size of the file. What happens if you point the program at `/dev/urandom`? What should the right behavior be?

3. Run `zig build -Doptimize=ReleaseFast run -- some_large_file`. Compare the timing against the default debug build. Chapter 17 explains why the gap is as large as it is.

4. Delete the `defer a.free(bytes)` line and rerun the program under the default build. What does the `GeneralPurposeAllocator` report? What does it report under `ReleaseFast`? Why?

In Chapter 2 we'll set up an environment that makes the code in this book reproducible on your machine. Then, in Chapter 3, we'll walk through the subset of Zig the rest of the book assumes you know — and no more.

CHAPTER

The Working Environment

The Zig toolchain is one binary. There is no package manager to install separately, no runtime to ship alongside your program, no build-system plugin to configure. Once `zig` is on your `PATH`, everything the book asks for — compiling, running, testing, packaging, cross-compiling, formatting, even the REPL-style `zig run` — is a subcommand of that one binary. This chapter walks through getting that binary onto your machine, pinning its version per project, and wiring up an editor that understands the code you write.

2.1 Installing Zig

The canonical distribution site is `ziglang.org/download`. Release tarballs are self-contained: unzip, drop the `zig` binary onto your `PATH`, and you are done. Pick a release that matches the version this book targets (0.14.x at the time of writing); later chapters label blocks that require a different version.

macOS

```
brew install zig
```

If you need a specific version:

```
brew install zig@0.14
```

If you want the master (development) build — useful if you are tracking a feature that has not landed in a release — install manually from the tarball and keep it out of `/usr/local` so that Homebrew does not shadow it.

Linux

```
curl -L https://ziglang.org/download/0.14.0/zig-linux-x86_64-0.14.0.tar.xz | tar -xJ
sudo mv zig-linux-x86_64-0.14.0 /opt/zig-0.14
echo 'export PATH="/opt/zig-0.14:$PATH"' >> ~/.bashrc
```

Distribution packages (`apt` , `dnf` , etc.) tend to trail the upstream release by a version or two. Prefer the tarball unless your platform happens to ship something current.

Windows

Download the `x86_64-windows` zip, extract it, and add the extracted directory to `PATH` . On Windows I recommend you work from a real terminal emulator (Windows Terminal) and not a Cygwin-style shell; all of Zig's tooling runs fine under cmd.exe and PowerShell, and the build system does not expect a POSIX environment.

Verifying

```
zig version
```

```
0.14.0
```

Then:

```
zig env
```

```
...
 "target": "x86_64-linux-gnu",
 "zig_exe": "/opt/zig-0.14/zig",
 "std_dir": "/opt/zig-0.14/lib/std",
...
```

`zig env` is worth remembering. It tells you which compiler is being picked up, where the standard library lives (useful when you want to read its source, which you will), and where caches are stored.

2.2 Pinning a compiler per project

A problem with tools that ship as single binaries is that every member of your team can end up on a different version. For most projects this is fine; for projects that use a language feature added recently, it is not.

The standard solution is `zig-version.txt` plus a trivial shell wrapper, but a better one — and the one used by projects like Bun and Mach — is `zigup` or `zvm`, two version-manager tools that behave like `nvm` for Node or `rustup` for Rust. `zvm` is the more maintained of the two at the time of writing:

```
# Install
curl -sSL https://github.com/tristanisham/zvm/raw/main/install.sh | bash

# Pin the project
echo "0.14.0" > .zigversion
zvm i 0.14.0
zvm use 0.14.0
```

With `.zigversion` committed, anyone who runs `zvm use` in the directory gets exactly the compiler the project was written against.

> **Tip**
> If you are just starting a single-person project, skip the version manager and commit a one-line `Makefile` target: `zig: ; curl -sL https://ziglang.org/download/.../zig-0.14.tar.xz | tar -xJ`. Add complexity only when it pays for itself.

2.3 The project layout

`zig init` produces the canonical layout:

```
hello/
■■■ build.zig
■■■ build.zig.zon
■■■ src/
    ■■■ main.zig
    ■■■ root.zig
```

- `build.zig` is the build script. It is a Zig program, not a configuration file — Chapter 13 is a whole chapter on it. For now you will mostly edit the target binary's name and source path at the top of the file.
- `build.zig.zon` is a dependency manifest in Zig Object Notation — a small, typed sibling of JSON. It lists your package's dependencies and their hashes. `zon` is short for "Zig object notation".
- `src/main.zig` is the entry point of an executable target.
- `src/root.zig` is the entry point of a library target (if you have one). The default template includes both so you can grow into either shape.

Everything else you add — tests, benchmarks, extra binaries — lives under `src/` or a sibling directory and is named in `build.zig`.

2.4 Running, testing, packaging

Four commands carry you through 95% of daily work:

```
zig build            # compile (debug mode) into zig-out/bin/
zig build run        # compile and run
zig build test       # run every test block in every file reachable
                     # from build.zig's test step
zig build -Doptimize=ReleaseFast   # optimized build
```

Release modes are worth naming:

- **Debug** (default): safety checks on, assertions on, no optimization. Use during development; your compile times and runtime both suffer, but you get stack traces and bounds checks.

- **ReleaseSafe**: optimized, but with safety checks still enabled. The best choice for most production server code.
- **ReleaseFast**: optimized and safety checks off. Use for anything measured in microseconds.
- **ReleaseSmall**: optimized for binary size. Use for embedded targets and distribution.

We will come back to these in Chapter 17. For now, just remember that *the default is debug*, which is slow and large, and that you need to ask for an optimized build explicitly.

The standalone runner

Outside a project, `zig run foo.zig` compiles and runs a single file — useful for quick experiments:

```
cat > /tmp/hi.zig <<'EOF'
const std = @import("std");
pub fn main() void {
    std.debug.print("standalone!\n", .{});
}
EOF

zig run /tmp/hi.zig
```

```
standalone!
```

`zig test foo.zig` does the same for tests.

2.5 Editor setup

The Zig Language Server (ZLS) is a separate binary that most editors pick up automatically once it is on your `PATH`. Install it to match your compiler version:

```
# with zvm
zvm i-zls 0.14.0

# or manually
git clone https://github.com/zigtools/zls && cd zls
zig build -Doptimize=ReleaseSafe
# drop ./zig-out/bin/zls onto your PATH
```

VS Code

Install the *Zig Language* extension. It will pick up `zls` from `PATH` and turn on go-to-definition, inline errors, formatting-on-save, and hover types. The one setting worth toggling is `"zig.formattingProvider": "zls"` — the default uses `zig fmt`, which is fine, but routing formatting through the language server is faster on big files.

Neovim

Neovim's built-in LSP client works out of the box. A minimal config:

```
require("lspconfig").zls.setup({
  settings = {
    zls = { enable_inlay_hints = true, enable_snippets = true },
  },
})
```

Pair with `treesitter` for syntax highlighting and `conform.nvim` for auto-formatting on save.

JetBrains

Use the *Zig* plugin (currently maintained by the Zig community). It wraps the same ZLS binary under the hood. Some JetBrains refactoring features (rename, extract method) are not yet implemented because the language server does not expose them; plain editing and error diagnostics do work.

2.6 Formatting and style

`zig fmt` is the canonical formatter and it has *no options*. Run it over your tree:

```
zig fmt src/
```

Editor integrations call this for you on save. There is no style argument; the community has decided that the cost of bikeshedding over brace placement is higher than the cost of everyone using the same brace placement. This book is formatted by `zig fmt` verbatim, with one small exception — the example in Chapter 3 uses tight line breaks that `zig fmt` would unroll, and I have kept them tight for print.

2.7 Where the standard library lives

Once in a while — once or twice a week, once you are fluent — you will want to read the standard library. It is unusually well-written code and the fastest way to answer questions like "how does `ArrayList` resize?" or "what error does this function return?".

```
$(zig env | grep std_dir | cut -d'"' -f4)
```

or, in `zig` itself:

```
zig env | zig run -e 'const std = @import("std"); const j = try std.json.parseFromSl
```

On macOS with Homebrew, that path is typically `/opt/homebrew/Cellar/zig/0.14.0/lib/std`. Open the directory in your editor and keep it bookmarked. When the docs and ZLS don't answer a question, the source always does.

> **Note**
> Zig's standard library is intentionally small compared to Go's or Python's. It includes the things the compiler itself needs, plus the things the community has agreed are load-bearing: memory allocators, data structures, file I/O, process management, basic networking, JSON. For anything beyond that — HTTP clients, TLS, SQLite — you reach for a third-party package (Chapter 13).

2.8 Dependency management

`build.zig.zon` is the manifest file, and `zig fetch` is the tool that mutates it:

```
zig fetch --save=uuid https://github.com/dmgk/zig-uuid/archive/refs/tags/v0.2.1.tar.
```

This downloads the tarball, verifies a hash, adds an entry to `build.zig.zon`, and caches the unpacked source under `~/.cache/zig/p/`. You then refer to the dependency from `build.zig`:

```
const uuid = b.dependency("uuid", .{});
exe.root_module.addImport("uuid", uuid.module("uuid"));
```

From your Zig code, `@import("uuid")` now resolves. Chapter 13 walks through the full build-system API; for now, remember that dependencies are files, not opinions — the hash in `build.zig.zon` pins the bytes, not the version, so a rebuild on another machine gets bit-identical sources.

2.9 Examples

Example A: a debug-friendly hello

```
const std = @import("std");

pub fn main() !void {
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer if (gpa.deinit() == .leak) @panic("memory leaked");
    const a = gpa.allocator();

    const msg = try std.fmt.allocPrint(a, "hello from Zig {s}\n", .{"0.14"});
    defer a.free(msg);

    try std.io.getStdOut().writer().writeAll(msg);
}
```

Running under the default debug mode turns on the leak detector, the bounds checker, and the stack-trace printer. Every program in this book is written with the assumption that you run it like this first.

Example B: a watched rebuild loop

```
find src -name '*.zig' | entr -c zig build test
```

Install `entr` (`brew install entr`; on Linux, `apt install entr`) and pipe your source files into it. Every save triggers a fresh `zig build test`, which gives you a live error list as you edit. The first rebuild is slow; subsequent rebuilds hit the cache and finish in milliseconds.

2.10 Exercises

1. Create a new project with `zig init`, then change `build.zig` so that `zig build run` runs a second binary `src/other.zig` instead of the default. Hint: copy the `b.addExecutable(...)` block.

2. Install two Zig versions side by side with `zvm`. Confirm that switching versions changes `zig version` output and that a project's `.zigversion` pins the correct one. Why does Zig ship as one binary?

3. Find `std.ArrayList` in the standard library. How many lines of code is it? (`wc -l`). What does that tell you about the language?

4. Run `zig build -Doptimize=ReleaseFast` on the word-count program from Chapter 1 and compare the binary size to the default debug build. Chapter 17 explains the gap.

In Chapter 3 we'll walk through the part of the Zig language you need to read the rest of the book — values, types, control flow, functions — and nothing you don't.

CHAPTER

Core Language, Fast

This chapter is the one where I tell you, in a rush, the parts of Zig you need to read everything that comes after. It is not a reference — the Zig language reference is thirty pages and very clear, and you should skim it at some point — but a concentrated tour of the expressions and statements we will use in the rest of the book. If you have written C recently, most of this will feel familiar. A few things will not, and those are the ones to read carefully.

3.1 Values and variables

Every variable is either `const` or `var`. `const` is a value you cannot rebind; `var` is one you can.

```
const pi: f64 = 3.14159265358979;
var count: usize = 0;
count += 1;
```

Types go after the name, separated by a colon. Most of the time you can leave the type off and let the compiler infer it:

```
const pi = 3.14159265358979;    // comptime_float
var count: usize = 0;           // type kept to force a concrete size
```

`const` is the default mentally — if you find yourself reaching for `var`, pause and ask whether the value actually needs to change. The compiler will also nudge you: unused mutations on a `var` produce a warning.

> **Warning**
> Local variables must be initialised. A `var x: i32;` without an initializer will not compile. If you truly want an uninitialized value (to fill in shortly), use `var x: i32 = undefined;` — this is honest, legal, and searchable.

3.2 Primitive types

Zig's numeric vocabulary is large and consistent:

Category

Types

Signed

`i8`, `i16`, `i32`, `i64`, `i128`, `isize`

Unsigned

`u8`, `u16`, `u32`, `u64`, `u128`, `usize`

Arbitrary

`i3`, `u13`, `i7` — any width from 0 to 65,535

Floats

`f16`, `f32`, `f64`, `f80`, `f128`

Booleans

`bool`

Byte

`u8` (aliased to `c_char` in interop contexts)

Void/noreturn

`void`, `noreturn`

The arbitrary-width types are a delight once you have them. If you are packing eight three-bit fields into a `u24`, the field type is `u3`, and the compiler does the right thing:

```
const Flags = packed struct(u8) {
    mode: u3,
    color: u2,
    bank: u3,
};
```

We will come back to packed structs in Chapter 6.

Integer overflow

A pleasant surprise from C: overflow is a *defined* checked error in debug builds and becomes undefined behavior in release. If you want wrapping, you say so explicitly:

```
var a: u8 = 250;
a +%= 10;           // wraps to 4
a |= 1;             // bitwise, always well-defined
const b = a +| 200; // saturates at u8 max (255)
```

The operators are:

- `+ - * /` — checked; traps in debug, UB in release
- `+% -% *%` — wrapping
- `+| -| *|` — saturating

This is the subtle pattern the language repeats everywhere: the default is the safe thing, and the unsafe thing has a visible syntactic mark.

Boolean operators

`and` , `or` , and `not` are keywords, not symbols. `!` is reserved for error unions. `==` and `!=` are equality; `<` , `<=` , `>` , `>=` are the rest of the comparisons. There are no truthy/falsy coercions: the condition in `if` , `while` , and `switch` must be a `bool` , an optional, or an error union.

3.3 Control flow

if

```
if (count == 0) {
    std.debug.print("empty\n", .{});
} else if (count < 10) {
    std.debug.print("small\n", .{});
} else {
    std.debug.print("lots\n", .{});
}
```

`if` is also an expression, which means it can produce a value:

```
const label = if (count == 0) "empty" else "non-empty";
```

Over an optional, `if` can bind the unwrapped value:

```
if (find(path)) |entry| {
    std.debug.print("found: {s}\n", .{entry.name});
} else {
    std.debug.print("missing\n", .{});
}
```

Over an error union, it can bind both sides:

```
if (parse(src)) |n| {
    std.debug.print("parsed {d}\n", .{n});
} else |err| {
    std.debug.print("failed: {s}\n", .{@errorName(err)});
}
```

switch

`switch` is exhaustive and is also an expression:

```
const side = switch (coin_flip) {
    .heads => "H",
    .tails => "T",
};
```

Ranges and multiple match values work too:

```
const bucket = switch (age) {
    0...12 => "child",
    13...19 => "teen",
    20...64 => "adult",
    else => "senior",
};
```

If the input is an enum, listing `else` when you have already covered every variant is a compile error — the language wants you to know when you have forgotten a case.

while, for

`while` is a loop with an optional continue expression:

```
var i: usize = 0;
while (i < 10) : (i += 1) {
    std.debug.print("{d}\n", .{i});
}
```

The `: (i += 1)` runs at the bottom of each iteration, even if you `continue`. It is a small, pleasant feature: you put the step with the condition, and the loop body does not have to remember to increment.

`for` iterates over slices and ranges:

```
const xs = [_]u32{ 1, 2, 3 };
for (xs) |x| std.debug.print("{d}\n", .{x});

for (xs, 0..) |x, i| {
    std.debug.print("[{d}] = {d}\n", .{ i, x });
}
```

The `xs, 0..` form zips a slice against a counting range; you can add as many paired slices as you like as long as they have the same length.

Both loops can have an `else` clause that runs when the loop *exits normally* (the condition became false or the range was exhausted). If the loop is broken out of, the `else` does not run. It sounds baroque and then you use it twice and it becomes your favorite feature:

```
const found = for (xs) |x| {
    if (x == target) break x;
} else null;
```

Blocks as expressions

Every block is an expression. A labeled block with a `break :label value` produces the value:

```
const max = blk: {
    var m: u32 = 0;
    for (xs) |x| if (x > m) { m = x; };
    break :blk m;
};
```

Labels let you break out of nested loops cleanly:

```
outer: for (rows) |row| {
    for (row) |cell| {
        if (cell == target) break :outer;
    }
}
```

3.4 Functions

Functions declare parameter and return types explicitly:

```
fn square(x: u32) u32 {
    return x * x;
}

pub fn main() void {
    std.debug.print("{d}\n", .{square(7)}); // 49
}
```

`pub` makes a function visible outside its file. Inside a file every declaration is private by default — a detail that matters more than it looks like it does, because it lets the compiler aggressively inline file-local helpers.

Parameters

Function parameters are by default passed by value for small (register-sized) types and as const pointers for larger ones. You cannot observe the difference from inside the function — every parameter is immutable — so you do not need to think about it. If you need to mutate through a parameter, take a pointer explicitly:

```
fn addOne(n: *u32) void {
    n.* += 1;
}

pub fn main() void {
    var x: u32 = 1;
    addOne(&x);
    // x == 2
}
```

Multiple returns

Zig does not have tuple-returns. When you want to return several values, return a struct:

```
const Point = struct { x: f64, y: f64 };

fn midpoint(a: Point, b: Point) Point {
    return .{ .x = (a.x + b.x) / 2, .y = (a.y + b.y) / 2 };
}
```

The `.{ ... }` syntax is an *anonymous struct literal*: the compiler infers the struct type from context — here, the function's return type. You will use this constantly.

Error-returning functions

The return type `!T` is an error union: "either a `T` or an error from some set". The error set is inferred by default, or you can name one:

```
const ParseError = error{ InvalidCharacter, Overflow };

fn parseU32(s: []const u8) ParseError!u32 {
    var acc: u32 = 0;
    for (s) |c| {
        if (c < '0' or c > '9') return ParseError.InvalidCharacter;
        acc = try std.math.mul(u32, acc, 10);
        acc = try std.math.add(u32, acc, c - '0');
    }
    return acc;
}
```

`try` unwraps the error union, short-circuiting with the error if there is one. Chapter 10 is about error handling in detail; for now, remember that `try` is to errors what `.?` is to optionals.

3.5 Optionals

`?T` is "either a `T` or null". There is no "null pointer exception"; the compiler refuses to let you use an optional without unwrapping it:

```
var maybe: ?u32 = 41;
// const n: u32 = maybe;             // compile error
const n: u32 = maybe orelse 0;       // n == 41
const m: u32 = (maybe orelse 0) + 1; // m == 42
```

`orelse` supplies a default, `if (x) |v|` binds the inner value conditionally, and `.?` unwraps and panics on null:

```
const v = maybe.?; // panics if maybe is null
```

I reach for `.?` about once a month. `orelse` and `if` handle almost every case, and they make the null handling visible at the call site.

3.6 Strings

There is no `string` type. A string literal is a sentinel-terminated array:

```
const greeting = "hello";
// @TypeOf(greeting) == *const [5:0]u8
```

That reads: "pointer to a length-5 array of u8 terminated by a 0 sentinel". In practice you rarely care about the exact type; most string-accepting functions take `[]const u8`, and a string literal coerces to it:

```
fn shout(s: []const u8) void {
    for (s) |c| std.debug.print("{c}", .{std.ascii.toUpper(c)});
    std.debug.print("\n", .{});
}

shout("hello"); // HELLO
```

UTF-8 in, UTF-8 out. Zig does not have a "unicode string" type — a string is bytes. `std.unicode` provides iterators over code points when you need them.

3.7 Arrays, slices, and pointers — a preview

A full treatment is in Chapter 4. For now, the short version:

```
const xs = [_]u32{ 1, 2, 3 };   // inferred-length array, type [3]u32
const ys: [3]u32 = .{ 4, 5, 6 }; // explicit-length array
const s = xs[0..2];              // slice, type []const u32

const p: *const u32 = &xs[0];    // single-item pointer
const m: [*]const u32 = &xs;     // many-item pointer (C-style)
```

The three pointer shapes exist because they mean different things: `*T` is "exactly one `T`"; `[*]T` is "an unknown number of `T`"; `[]T` is "a known run of `T`, which I am carrying the length around with". `[]T` is what you will use almost always.

3.8 Imports and modules

A file is a struct. `@import("x")` returns that struct:

```
// parse.zig
pub fn parseU32(s: []const u8) !u32 { ... }
pub fn parseF64(s: []const u8) !f64 { ... }
```

```
// main.zig
const parse = @import("parse.zig");

pub fn main() !void {
    const n = try parse.parseU32("42");
    ...
}
```

`@import("std")` pulls in the standard library by name — the build system knows where to find it. Third-party packages are added via `build.zig` (Chapter 13) and imported the same way:

```
const uuid = @import("uuid");
```

3.9 A warm-up program

Let's stitch these pieces into something real: a program that reads numbers from stdin and prints the mean and variance.

```zig
const std = @import("std");

pub fn main() !void {
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer _ = gpa.deinit();
    const a = gpa.allocator();

    const stdin = std.io.getStdIn().reader();
    const raw = try stdin.readAllAlloc(a, 1 << 20);
    defer a.free(raw);

    var xs = std.ArrayList(f64).init(a);
    defer xs.deinit();

    var it = std.mem.tokenizeAny(u8, raw, " \t\n\r,");
    while (it.next()) |tok| {
        const v = std.fmt.parseFloat(f64, tok) catch |err| {
            std.debug.print("skipping '{s}': {s}\n", .{ tok, @errorName(err) });
            continue;
        };
        try xs.append(v);
    }

    if (xs.items.len == 0) {
        std.debug.print("no numbers\n", .{});
        return;
    }

    var sum: f64 = 0;
    for (xs.items) |v| sum += v;
    const mean = sum / @as(f64, @floatFromInt(xs.items.len));

    var ss: f64 = 0;
    for (xs.items) |v| {
        const d = v - mean;
        ss += d * d;
    }
    const variance = ss / @as(f64, @floatFromInt(xs.items.len));

    const w = std.io.getStdOut().writer();
    try w.print("n={d}  mean={d:.4}  var={d:.4}\n",
        .{ xs.items.len, mean, variance });
}
```

```
$ printf "1 2 3 4 5\n" | zig build run
n=5  mean=3.0000  var=2.0000
```

Every Zig idiom this book relies on is in this program: an allocator at the top, `defer` cleanup in reverse order, `try` on every fallible call, `catch |err|` for a specific fallible call where we do not want to abort, a dynamic array from the standard library, a tokenizing iterator, and a typed cast (`@floatFromInt`) where the language forces you to be explicit about how the conversion happens.

Notice what is *not* there: no exception, no finalizer, no hidden heap allocation during the number-parsing loop (tokenization is zero-allocation), and no way for the program to leave scope with leaked memory.

3.10 Exercises

1. Extend the warm-up program to also print the min and max. The standard library has `std.mem.min` and `std.mem.max`; find them in `std/mem.zig` and use them. What is their signature?

2. Change the error handling on `std.fmt.parseFloat` from "warn and continue" to "stop and propagate". Now a single bad token terminates the program — show that by running with a file that has `oops` in the middle.

3. Replace the `std.ArrayList(f64)` with a fixed-size `[1024]f64` buffer and a `usize` counter. What happens if the input has more than a thousand numbers? Whose responsibility is it to catch the error — the compiler's, the programmer's, or the user's?

4. Rewrite `parseU32` (from earlier in this chapter) using only bitwise operations on `u8`s. What size inputs does it accept correctly? What does it do on overflow? (Chapter 10 covers the right answer.)

With this, we have enough of the language to talk seriously about memory. Chapter 4 — values, pointers, and slices — is where everything you've seen so far starts to matter.

CHAPTER

Values, Pointers, Slices

The single largest source of confusion when people come to Zig from a higher-level language is the difference between an array, a slice, and the several kinds of pointer. In a language like Python or Go there is really only one shape: a list, a string, a slice — the distinction between "the data" and "a handle to the data" is blurred, because the runtime is carrying the distinction for you. Zig does not carry it for you. The payoff for learning where the distinction lives is that you will never be surprised by a copy, an alias, or a dangling reference.

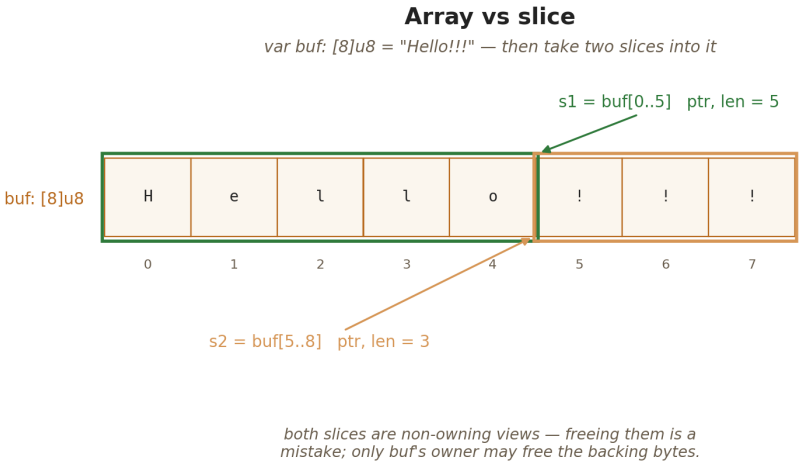
This chapter is twenty pages on that one distinction, with pictures.

4.1 The three shapes

Zig has three fundamentally different ways of representing a "bunch of `T`s in a row":

- **Array:** `[N]T` — a value type of compile-time-known length.
- **Many-item pointer:** `[*]T` — a bare pointer to zero or more `T`s. You carry the length (if any) separately.
- **Slice:** `[]T` — a pointer-length pair. The workhorse.

All three exist because they mean different things to the compiler. An array *is* the values, stored inline; a many-item pointer is an address with no bounds information; a slice is an address *plus* a runtime length, which is almost always what you want.



An array stores the bytes; a slice is a view into them. Two slices can refer to the same (or overlapping) runs of bytes. Freeing a slice is a bug — only the owner of the backing memory may free it.

4.2 Arrays

An array literal looks like this:

```
const xs = [3]u32{ 1, 2, 3 };
const ys = [_]u32{ 4, 5, 6 }; // length inferred from the initializer
```

The `[N]T` type carries its length in the type. That has two consequences:

- Indexing past the end is a *compile error* when the index is known at compile time; at runtime, it is a checked bounds-check failure in debug/safe builds.
- The size of an array value is exactly `N * @sizeOf(T)` bytes — there is no header, no length word, nothing. Arrays are POD.

The array's length is a compile-time constant you can ask for:

```
const n = xs.len;           // comptime-known to be 3
const t = @TypeOf(xs);      // the type [3]u32
const e = @typeInfo(t).array.len; // also 3
```

You pass arrays *by value*. A function that takes `[3]u32` gets its own copy:

```
fn sum(a: [3]u32) u32 {
    return a[0] + a[1] + a[2];
}
```

This is efficient when `N` is small; it is a copy when `N` is large. If you want to share instead of copy, you want a slice, not an array.

4.3 Slices

A slice is a pair — a pointer to the first element and a runtime length. Its type is `[]T` (immutable view) or `[]const T` (read-only view) — the difference lives in whether you can write through the pointer.

You make a slice by taking a range of an array:

```
const xs = [_]u32{ 10, 20, 30, 40, 50 };
const s: []const u32 = xs[1..4]; // [20, 30, 40]
std.debug.print("len={d} first={d}\n", .{ s.len, s[0] });
```

```
len=3 first=20
```

`xs[1..4]` builds a slice whose pointer is `&xs[1]` and whose length is `4 - 1 = 3`. The slice is a *view*: it does not own the memory it points to. The bytes still live in the array `xs`. If the array goes out of scope, the slice points into garbage; the compiler will not catch that, and neither will the runtime.

Two consequences:

- You can have many slices over the same backing memory, and they can overlap.
- Passing a slice is cheap — it is a pointer-plus-length, two machine words — and does *not* copy the underlying data.

> **Warning**
> A slice does not own its bytes. Whoever allocated them is responsible for freeing them. If you return a slice from a function, you are returning an alias to something — make sure you know what.

Slice operations

The slice grammar is small:

```
s[0..3]    // sub-slice, half-open range [0, 3)
s[2..]     // sub-slice from index 2 to the end
s[0..s.len] // equivalent to s, just with len re-checked at runtime
```

Slices have three fields you can reach:

- `s.ptr` — a many-item pointer to the first element (`[*]T`).
- `s.len` — the runtime length (`usize`).

Iteration is a `for`:

```
for (s) |v| { ... }            // values
for (s, 0..) |v, i| { ... }     // values and indices
for (s) |*v| { v.* += 1; }      // mutable references
```

A loop over a slice does a bounds check exactly once, at entry — not on every index.

4.4 Pointers, properly

Zig has four pointer types, which sounds like a lot until you see that each one has a different job:

- `*T` — *single-item* pointer. Exactly one `T`, guaranteed non-null.
- `?*T` — optional single-item pointer. One `T` or null.
- `[*]T` — *many-item* pointer. One or more `T`s; no bounds. C's `T*`, essentially.
- `[*c]T` — *C* pointer. Like `[*]T`, but coerces to/from `?*T` and `*T` — used exclusively when translating C headers.

Single-item pointers

`*T` is what you take with `&`:

```
var x: u32 = 7;
const p: *u32 = &x;
p.* = 8;
// x == 8
```

The `.*` is the dereference operator. You will also see field access through pointers:

```
const Point = struct { x: f64, y: f64 };
var pt = Point{ .x = 1, .y = 2 };
const pp = &pt;
pp.x = 10; // same as pp.*.x = 10; the .* is implicit for structs
```

The `pp.x = 10` form is a small ergonomic favor; the compiler inserts the dereference. Every other operation — method calls, arithmetic — works the same way.

Optional pointers

A `*T` is never null. If you want nullability, say so:

```
fn find(name: []const u8) ?*const Entry { ... }

if (find("foo")) |e| {
    // e: *const Entry, guaranteed non-null
    std.debug.print("found {s}\n", .{e.name});
}
```

Because `?*T` uses the same bit pattern for "null" as a C null pointer, it has no runtime overhead — the compiler just tracks the nullability in the type system and makes you unwrap before use.

Many-item pointers

`[*]T` is the C-style "pointer to first of some run of `T`s". You rarely write it directly; it mostly shows up when you interop with C, or when you want to turn a slice into a bare pointer for a native function that cannot take a slice:

```
extern fn memcpy(dst: [*]u8, src: [*]const u8, n: usize) void;

pub fn copy(dst: []u8, src: []const u8) void {
    std.debug.assert(dst.len >= src.len);
    memcpy(dst.ptr, src.ptr, src.len);
}
```

In practice you will wrap `[*]T` with a slice (`dst.ptr`, `src.ptr`) and let the slice carry the length.

Pointer arithmetic

Pointer arithmetic is legal only on many-item pointers and C pointers:

```
var xs = [_]u32{ 1, 2, 3, 4 };
var p: [*]u32 = &xs;
p += 1;                 // now points to xs[1]
std.debug.print("{d}\n", .{p[0]}); // 2
```

This is a sharp edge, which is why `[*]T` is rarely your first choice. Slices give you bounds, `*T` gives you bounded single-element pointers; reach for `[*]T` only when you need exactly what it offers.

4.5 Sentinel-terminated types

A whole family of Zig types carries an extra element at the end as a sentinel value. The most common is `[:0]const u8` — a slice of bytes whose last element is guaranteed to be zero — which is how Zig interoperates with C strings:

```
const c_string: [:0]const u8 = "hello";
```

You can also have sentinel-terminated arrays and many-item pointers:

```
const lit: *const [5:0]u8 = "hello"; // array, len 5, sentinel 0
const ptr: [*:0]const u8 = "hello";   // no length, scan to sentinel
```

When you pass a Zig string literal into a C function, the coercion rules pick the right one:

```
extern fn puts(s: [*:0]const u8) c_int;

pub fn main() void {
    _ = puts("hello via libc");
}
```

The sentinel-terminated types are a nice middle ground between "I know the length at compile time" and "I know nothing at all" — they let the C world's conventions live inside the Zig type system.

4.6 View vs copy: a small drill

You are reviewing code. Which of these copies, and which creates a view?

```
const a = [_]u32{ 1, 2, 3 };
const b = a;              // copy of the array (value type)
const c: []const u32 = &a; // view (slice over the same bytes)
const d = a[0..];         // view
var e = a;                // mutable copy
e[0] = 99;                // a is still { 1, 2, 3 }
```

The rule: an array assignment copies; a slice assignment is a pointer+length assignment. `&a` coerces a pointer-to-array into a slice of the array's bytes.

A function argument of type `[3]u32` is a copy; `[]const u32` is a view. Prefer slices at API boundaries unless you specifically want the copy (rare).

4.7 Passing mutably

Most things in this book will be passed by `const` slice. When you need to mutate through the function call, there are two common idioms. The first is a plain pointer for scalars:

```
fn incr(p: *u32) void { p.* += 1; }
```

The second is a mutable slice for ranges:

```
fn zero(buf: []u8) void {
    for (buf) |*b| b.* = 0;
}

// or, with the stdlib helper:
fn zero2(buf: []u8) void { @memset(buf, 0); }
```

Note `for (buf) |*b|`: the `*b` binds a mutable pointer to each element, so `b.*` writes through it. Without the asterisk you get the value, not a reference, and the write would not stick.

4.8 Optionals, again

`?T` is a tagged union of "present" and "absent". The compiler uses the spare bit pattern (`null`) efficiently: for optional pointers, it reuses the zero address; for optional slices, it reuses a zero-length slice; for other optionals, it adds one bit plus alignment padding. You do not normally need to think about the representation, but you can check it:

```
@sizeOf(?u32)   // 8 on most 64-bit targets (4 bytes value + 4 padding)
@sizeOf(?*u32)  // 8 — the null-pointer optimization
@sizeOf(?[]u8)  // 16 — same as []u8, because len=0 is "null"
```

The canonical patterns for consuming an optional are:

```
// explicit check
if (maybe) |v| useIt(v);

// default
const v = maybe orelse 0;

// panic (avoid except in tests and prototypes)
const v = maybe.?;

// extract-and-replace
const old = maybe;
maybe = null;
```

4.9 Worked example: a ring buffer

Let's put the pieces together. A ring buffer stores at most `N` elements in a fixed-size array; writes overwrite the oldest entries:

```zig
const std = @import("std");

fn RingBuffer(comptime T: type, comptime N: usize) type {
    return struct {
        buf: [N]T = undefined,
        head: usize = 0,
        len: usize = 0,

        const Self = @This();

        pub fn push(self: *Self, v: T) void {
            const slot = (self.head + self.len) % N;
            self.buf[slot] = v;
            if (self.len < N) self.len += 1 else self.head = (self.head + 1) % N;
        }

        pub fn get(self: *const Self, i: usize) T {
            std.debug.assert(i < self.len);
            return self.buf[(self.head + i) % N];
        }

        pub fn asSlices(self: *const Self) [2][]const T {
            if (self.len == 0) return .{ &.{}, &.{} };
            const end = (self.head + self.len) % N;
            if (end > self.head) {
                return .{ self.buf[self.head..end], &.{} };
            }
            return .{ self.buf[self.head..], self.buf[0..end] };
        }
    };
}

test "ring buffer" {
    var rb = RingBuffer(u8, 3){};
    for ([_]u8{ 'a', 'b', 'c', 'd' }) |c| rb.push(c);
    try std.testing.expectEqual(@as(u8, 'b'), rb.get(0));
    try std.testing.expectEqual(@as(u8, 'd'), rb.get(2));
}
```

The things to notice:

1. `RingBuffer(u8, 3)` returns a *type*. Zig's generics are just functions that return types at compile time — Chapter 8 is a whole chapter on this, but you already have enough to read the code.

2. `buf: [N]T = undefined` uses a default value of `undefined`. The ring buffer never reads a slot it has not written to, so leaving the initial contents undefined is correct and cheap.
3. `asSlices` returns *two* slices. Because the logical window wraps around the end of the backing array, a single contiguous slice cannot always represent it. Returning a pair of slices is the canonical Zig answer: both slices alias the same backing storage, and callers iterate them back-to-back.
4. The tests use `std.testing.expectEqual`. Test blocks run under `zig test` or `zig build test`; Chapter 12 is the full tour.

Run it:

```
$ zig test ring.zig
All 1 tests passed.
```

4.10 Examples

Example A: zero-copy CSV splitting

```
pub fn fields(line: []const u8) []const []const u8 {
    // (illustrative — the real version would take an allocator.
    //  This one demonstrates that you can build a []const []const u8
    //  whose inner slices are all views into `line`.)
    var buf: [16][]const u8 = undefined;
    var n: usize = 0;
    var start: usize = 0;
    for (line, 0..) |c, i| {
        if (c == ',') {
            buf[n] = line[start..i];
            n += 1;
            start = i + 1;
        }
    }
    buf[n] = line[start..];
    n += 1;
    return buf[0..n];
}
```

Every returned inner slice points into `line`. No allocations, no copies. The caller must make sure `line` outlives the returned slice.

Example B: a mutable view

```
pub fn to_upper(s: []u8) void {
    for (s) |*c| c.* = std.ascii.toUpper(c.*);
}
```

This takes a mutable slice and rewrites in place. If you pass in a slice over a string literal, the program will crash on the first write — literals live in `.rodata`, which is not writable. The type system does not help here: `[]u8` means "writable bytes", and coercing a literal to `[]u8` is technically possible via `@constCast`, which exists precisely because it is *always* a lie. Do not do it without a very good reason.

4.11 Exercises

1. Write `fn reverse(comptime T: type, s: []T) void` that reverses a slice in place. Test it on both `[]u8` and `[]i32`.

2. Given a `[]const u8` containing UTF-8, write a function that returns the number of code points. `std.unicode.utf8Decode` and `std.unicode.utf8ByteSequenceLength` are useful.

3. Change the ring buffer to use a `[*]T` instead of a `[N]T` so it can be constructed with a runtime-sized backing buffer. Which fields change? What now takes an allocator?

4. Explain, in one sentence, why `[]const T` and `*const [N]T` are different types even though they describe the same bytes. When would you prefer each?

In Chapter 5 we'll pick up the topic this chapter has been skirting — the allocator — and spend another twenty pages making it feel natural.

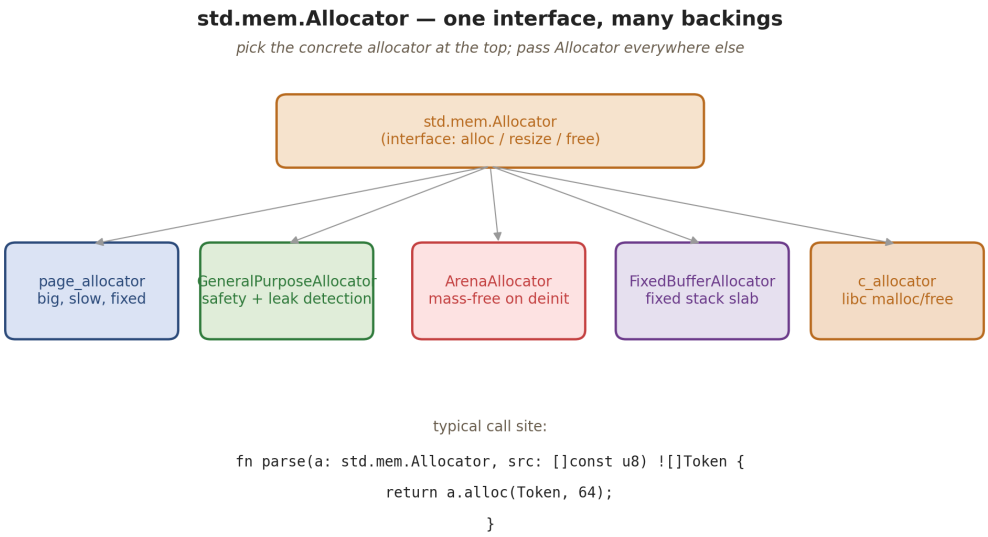
CHAPTER

Allocators

Zig does not have a default allocator. This is the single most unusual thing about the language, and the one that takes the longest to internalise. Every other topic in the book — slices, error handling, comptime, the build system — is a variation on an idea you have met before. Allocators are not. They ask you to think about memory the way systems programmers have always thought about memory, but to do so in plain sight: in the type signatures of the functions you call.

The reward, after a week of practice, is that your programs stop surprising you. A function that allocates has `Allocator` in its signature. A function that does not, does not. A function that owns memory is the one that calls `free` or `deinit`. There is no global heap, no implicit allocation, no "new" operator that conjures memory behind the scenes. This chapter is a tour of the idiom.

5.1 The interface



Every concrete allocator implements the same vtable. Pick one at the top of your program; everything below it takes an `Allocator` and does not know (or care) which kind it got.

`std.mem.Allocator` is a small struct:

```
pub const Allocator = struct {
    ptr: *anyopaque,
    vtable: *const VTable,

    pub const VTable = struct {
        alloc: *const fn (*anyopaque, usize, u8, usize) ?[*]u8,
        resize: *const fn (*anyopaque, []u8, u8, usize, usize) bool,
        free: *const fn (*anyopaque, []u8, u8, usize) void,
    };

    // ... high-level methods: alloc, alloc_sentinel, create, destroy, free, ...
};
```

You never call `alloc` on the vtable directly. Every allocator exposes a richer typed surface:

```
const xs = try a.alloc(u32, 64);      // slice of 64 u32s
defer a.free(xs);

const p = try a.create(MyStruct);      // one MyStruct on the heap
defer a.destroy(p);
```

`a.alloc(T, n)` returns `![]T`; `a.create(T)` returns `!*T`. The corresponding `free` / `destroy` calls release the memory. Mismatching them (creating with `create` and freeing with `free`) is a bug the compiler will not catch — you own the bookkeeping.

5.2 The concrete allocators

The standard library ships a handful of concrete allocators. Each answers a different question.

GeneralPurposeAllocator

The one you will use for most of this book. A thread-safe, leak-detecting, double-free-detecting allocator. It is slower than `page_allocator` and `c_allocator`, and that is the point: it tells you when you have made a memory mistake so that you can fix it.

```
var gpa = std.heap.GeneralPurposeAllocator(.{}){};
defer if (gpa.deinit() == .leak) @panic("leaked");
const a = gpa.allocator();
```

`gpa.deinit()` at the end of the program returns `.ok` or `.leak`; if it returns `.leak`, the allocator has already printed the offending stack traces to stderr. You get leak detection *for free*, just by using this allocator during development.

In production, swap it out for `page_allocator` or `c_allocator` — see Chapter 17.

> **Warning**
> `GeneralPurposeAllocator(.{})` is not a zero-cost abstraction. Its safety features cost cycles and memory; it is meant for development, tests, and "good enough for now" production. For hot paths, reach for an arena or `c_allocator`.

page_allocator

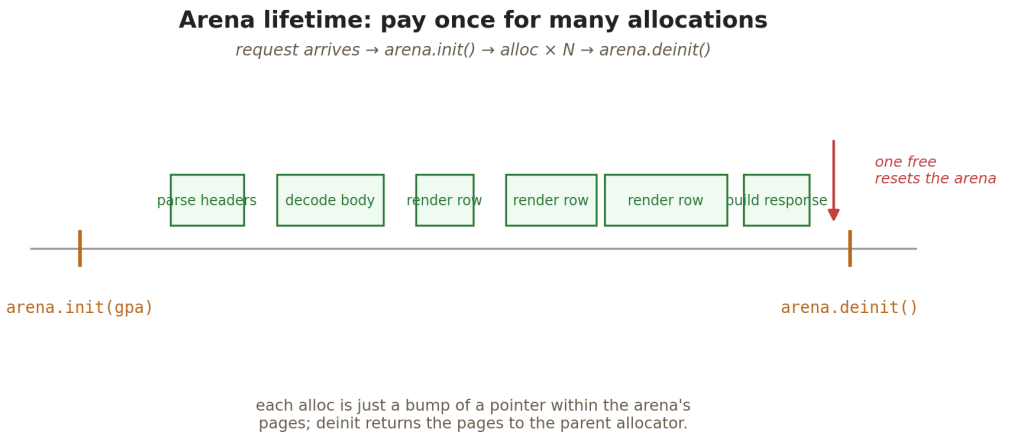
The OS page allocator (mmap on POSIX, VirtualAlloc on Windows). Every allocation is rounded up to a page boundary, so it is great for occasional large allocations and terrible for many small ones:

```
const a = std.heap.page_allocator;
const big = try a.alloc(u8, 4 * 1024 * 1024);
defer a.free(big);
```

There is no per-program state — `page_allocator` is a pre-made `Allocator` value. It is the usual *fallback* allocator that more specialized ones wrap.

ArenaAllocator

An arena is a bump-the-pointer allocator that frees nothing until you deinit it — at which point it frees everything at once. It is the Zig idiom for "a bunch of allocations that all have the same lifetime":



Pay once at the end. An arena is ideal when many small allocations share a single deinit point — a per-request allocator in a server, or a parse tree that is freed after use.

```
var arena = std.heap.ArenaAllocator.init(gpa.allocator());
defer arena.deinit();
const a = arena.allocator();

const buf1 = try a.alloc(u8, 1024);
const buf2 = try a.alloc(u32, 256);
// No individual frees; arena.deinit() releases everything.
```

The parent allocator (the argument to `init`) is where the arena gets its backing pages. An arena typically has two or three pages live at a time — it rounds up large allocations and batches small ones into the current page.

Use arenas when:

- You are processing a unit of work (an HTTP request, a parse tree) that has one beginning and one end.
- You are running a tool (compiler, build script) where you do not care about freeing until the whole program exits.
- You are in a test and want to say "don't bother cleaning up".

Avoid arenas when:

- Items have independent lifetimes — some allocated near the start and freed early, others allocated late and kept around. The arena holds on to everything, so you will effectively have a memory leak that grows with the arena's lifetime.
- You need to return data out of the arena's lifetime. (You can, of course, copy it to a longer-lived allocator before returning — and sometimes that is the right shape.)

FixedBufferAllocator

A fixed-size buffer, handed to you as an `Allocator`. It allocates by bumping an offset into the buffer; it fails when the buffer is full. It frees nothing.

```
var buf: [4096]u8 = undefined;
var fba = std.heap.FixedBufferAllocator.init(&buf);
const a = fba.allocator();

const xs = try a.alloc(u32, 32);
// ... use xs ...
fba.reset(); // reuse the buffer for the next batch
```

This is the allocator you reach for when you want to use the standard library's typed helpers (like `ArrayList`) but not pay for a heap allocation — perhaps on an embedded target, or in a hot loop where the lifetime is known and small.

c_allocator

A thin wrapper over `malloc` / `free`. Fast, no leak detection. Only usable if you link libc (`-lc`). Use it in production when you have already paid to link libc for some other reason.

```
const a = std.heap.c_allocator;
```

smp_allocator and thread-local caches

For multi-threaded programs, `std.heap.smp_allocator` is a lock-free, thread-cached allocator that is considerably faster than GPA under contention. It is the right default for server code.

5.3 The ownership discipline

Every allocated slice has *exactly one owner*. The owner is the piece of code responsible for the matching `free`. The owner can change — a function can *return* ownership to its caller — but at any moment there is exactly one. In Zig there is no runtime that tracks this; the programmer does. Two conventions make the tracking almost mechanical:

1. **A function that allocates takes an `Allocator` parameter.** A function without an `Allocator` parameter cannot allocate. Read the parameter list to know whether you need to free anything the function returns.
2. **The caller who provided the allocator owns what the function returns.** Idiomatic APIs are explicit about this in the doc comment and implicit in the signature: if you handed an allocator to the function, the slice it gives back is yours to free.

Two idioms tie a bow around this:

```
// "caller owns the returned slice"
pub fn readAll(a: Allocator, path: []const u8) ![]u8 {
    return std.fs.cwd().readFileAlloc(a, path, 1 << 24);
}

// "builder object that owns its contents until deinit"
pub fn build(a: Allocator) ![]u8 {
    var list = std.ArrayList(u8).init(a);
    defer list.deinit(); // defensively clean up on error
    try list.appendSlice("hello");
    try list.appendSlice(", ");
    try list.appendSlice("world");
    return list.toOwnedSlice(); // transfer ownership to the caller
}
```

`toOwnedSlice` is the textbook handoff. It returns the list's backing storage *and* empties the list, so the `defer list.deinit()` becomes a no-op and the caller is now responsible for `free`ing the returned slice with the same allocator.

5.4 errdefer: cleaning up on error only

`defer` runs at scope exit — normal or error. `errdefer` runs only when the scope exits with an error. It is the right tool for "tentatively acquire a resource, release it if anything after goes wrong":

```
pub fn init(a: Allocator) !Self {
    const buf = try a.alloc(u8, 1024);
    errdefer a.free(buf);

    const handle = try std.fs.cwd().openFile("log", .{});
    errdefer handle.close();

    try handle.writeAll("init\n");
    return .{ .buf = buf, .handle = handle, .allocator = a };
}
```

If `openFile` fails, the `errdefer a.free(buf)` runs and `buf` is released. If `writeAll` fails, *both* errdefers run, in reverse order (`handle.close()` then `a.free(buf)`). If everything succeeds, neither errdefer runs — both resources are now owned by the returned `Self` and will be freed in `deinit`.

We will see more of `errdefer` in Chapter 11. The reason it is introduced here is that allocator discipline is where most people first need it.

5.5 Choosing an allocator

A rule of thumb:

Situation

Allocator

Dev loop, unsure, want leak detection

`GeneralPurposeAllocator`

Per-request / per-unit-of-work, short-lived

`ArenaAllocator`

Embedded, no dynamic memory allowed

`FixedBufferAllocator`

Already linking libc, need speed

`c_allocator`

Huge one-off allocation

`page_allocator`

Hot multi-threaded server

`smp_allocator` or `c_allocator`

Test

`std.testing.allocator` (leak-detecting)

It is common for a program to use *several* allocators — a GPA for the lifetime of the program, an arena per request, and a fixed buffer for a hot loop. Zig's allocator discipline makes mixing cheap, because every function below the top is polymorphic over its caller's choice.

5.6 A worked example: per-request arena

Let's write a function that "processes a request" by parsing lines and building a response. The processing uses a lot of short-lived intermediate allocations; the response is the only thing we want to return to the caller.

```zig
const std = @import("std");
const Allocator = std.mem.Allocator;

pub const Response = struct {
    bytes: []u8,

    pub fn deinit(self: Response, a: Allocator) void {
        a.free(self.bytes);
    }
};

pub fn handle(parent: Allocator, req: []const u8) !Response {
    var arena = std.heap.ArenaAllocator.init(parent);
    defer arena.deinit();
    const a = arena.allocator();

    var words = std.ArrayList([]const u8).init(a);
    var it = std.mem.tokenizeAny(u8, req, " \t\n,");
    while (it.next()) |w| try words.append(w);

    var out = std.ArrayList(u8).init(parent);
    errdefer out.deinit();

    try out.writer().print("{d} words\n", .{words.items.len});
    for (words.items) |w| {
        try out.writer().print("- {s}\n", .{w});
    }

    return .{ .bytes = try out.toOwnedSlice() };
}

test "handle" {
    const a = std.testing.allocator;
    const resp = try handle(a, "the quick brown fox");
    defer resp.deinit(a);
    try std.testing.expect(std.mem.startsWith(u8, resp.bytes, "4 words"));
}
```

The structure is the Zig pattern in miniature:

1. Open an arena at the top of the unit of work.
2. Do all the scratch work with the arena's allocator.
3. Build the one output we want to keep with the *parent* allocator.
4. Return the output; the arena's `defer` releases everything else.

If `handle` errors anywhere in the middle, the `defer arena.deinit()` frees the scratch allocations and the `errdefer out.deinit()` frees the half-built output. The caller sees either a full `Response` or an error — never a partial build with leaks.

Run the test under `zig test`, and the `std.testing.allocator` will assert that every allocation was matched by a free. This combination — arena for scratch, explicit allocator for the result, and a leak- detecting test allocator — is the safety net Zig provides in place of a garbage collector.

5.7 Examples

Example A: a bounded work queue

```
var buf: [16 * 1024]u8 = undefined;
var fba = std.heap.FixedBufferAllocator.init(&buf);
const a = fba.allocator();

var queue = std.ArrayList(Task).init(a);
for (input) |t| try queue.append(t);
// ... process queue ...
```

Everything lives in `buf`. If `input` grows past ~2000 entries, the push will return `error.OutOfMemory` and your program finds out immediately. On embedded targets, this is the only legal shape.

Example B: the test allocator

```
test "no leaks" {
    const a = std.testing.allocator;
    const xs = try a.alloc(u32, 10);
    defer a.free(xs); // remove this line and the test fails
}
```

`std.testing.allocator` is a GPA wired to fail the current test on any leak. Use it in every test that touches memory — which is most of them.

5.8 Exercises

1. Rewrite the word-count program from Chapter 1 to use an arena for the parsing scratch state and the parent allocator only for the final output string. Confirm that under `std.testing.allocator` it reports no leaks.

2. Write a pool allocator: given a type `T` and a count `N`, pre-allocate `N` slots and let callers `get` and `put` with no further heap traffic. What is the signature? What does it do when the pool is empty?

3. Read the source of `ArenaAllocator` in the standard library. How does it choose the size of the next backing page? What happens when an allocation exceeds the page size?

4. Write a function that takes an `Allocator` and a `[]const []const u8` and returns a single `[]u8` containing every string joined by a separator. Who owns the returned slice? How do you document that?

In Chapter 6 we'll look at the aggregate types — structs, enums, unions, packed structs — that the allocator-aware APIs return and consume.

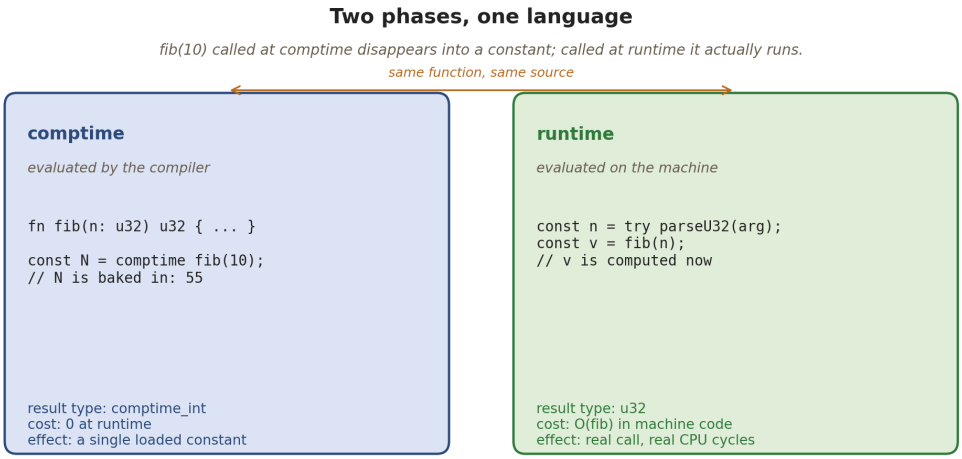
CHAPTER

comptime — The Short Version

The single most distinctive feature of Zig is that it has no separate template language. There is no macro system, no attribute DSL, no `constexpr` modifier, no preprocessor. Instead there is one good idea, repeated everywhere: *the compiler can run a subset of Zig at compile time, and a compile-time value can be of any type — including a type.* Every other language's solutions to generics, reflection, code generation, and constant folding are all, in Zig, the same mechanism.

This chapter is the short version — enough to read the rest of the book. Chapters 8 and 9 are the long versions, about generics and reflection respectively.

6.1 Two phases, one language



The same function can run in two places. A comptime call vanishes into a constant in the binary; a runtime call emits real code.

Every Zig program is compiled in two phases. The first phase *runs* (yes, runs — the compiler has an interpreter) the parts of your code marked `comptime`, plus any code those parts reach. The second phase emits machine code for whatever is left.

A function can be called in either phase. Consider:

```
fn fib(n: u32) u32 {
    if (n < 2) return n;
    return fib(n - 1) + fib(n - 2);
}
```

You can call `fib` the ordinary way, with a runtime argument:

```
const user_n = try parseU32(argv[1]);
const v = fib(user_n); // runs on the CPU
```

Or you can call it at compile time, with a compile-time argument:

```
const N = comptime fib(10); // evaluated by the compiler
// N is a constant in the binary: 55
```

The keyword `comptime` at the call site tells the compiler "please evaluate this expression during compilation". The function body is the same either way. The result type is a bit different — a comptime call can produce a `comptime_int`, a type whose size is not pinned until it is assigned to a typed variable — but we are going to ignore that subtlety until Chapter 8.

> **Note**
> Not every function is runnable at compile time. A function that dereferences a pointer of unknown provenance, or calls a syscall, or depends on a value that is not known until runtime, can't be. The compiler errors clearly when you ask for the impossible.

6.2 comptime parameters

If a function takes a parameter marked `comptime`, callers *must* supply a value known at compile time:

```
fn repeat(comptime T: type, value: T, n: usize) [n]T {
    var out: [n]T = undefined;
    for (out) |*slot| slot.* = value;
    return out;
}
```

Wait — that does not compile. `n` is a runtime value, so it cannot determine the array length. Fix it:

```
fn repeat(comptime T: type, value: T, comptime n: usize) [n]T {
    var out: [n]T = undefined;
    for (&out) |*slot| slot.* = value;
    return out;
}

const threes = repeat(u32, 3, 4);
// threes is [4]u32{ 3, 3, 3, 3 }
```

Now the type (`[4]u32`) is known at compile time, because `n` is.

Every time you see a `comptime` parameter, remember what it buys you: the compiler can use its value in *types*, array lengths, switch cases, and other places that otherwise require a constant.

6.3 Types are values

The most surprising sentence in this chapter: *types are first-class values at compile time*. You can pass them to functions, return them from functions, compare them, store them in arrays. You cannot do any of that with a type at runtime, because types have no runtime representation — they have been used up by the time the binary runs. But at compile time they are values like any other.

```
fn biggest(comptime types: anytype) type {
    var best = types[0];
    for (types[1..]) |t| {
        if (@sizeOf(t) > @sizeOf(best)) best = t;
    }
    return best;
}

const T = biggest(.{ u8, u32, u64, f32 }); // u64
```

The function is runtime-looking Zig, but it is running in the compiler. The `.{ ... }` is an anonymous struct / tuple — you pass a heterogeneous list of types and the function loops over them. The return type is `type`, which is legal only at compile time.

6.4 Generics as functions-that-return-types

The combination of "types are values" and "a function can take comptime parameters" gives you generics without any special syntax:

```
fn Stack(comptime T: type) type {
    return struct {
        items: []T,
        len: usize = 0,

        const Self = @This();

        pub fn init(a: std.mem.Allocator, cap: usize) !Self {
            return .{ .items = try a.alloc(T, cap) };
        }
        pub fn deinit(self: *Self, a: std.mem.Allocator) void {
            a.free(self.items);
        }
        pub fn push(self: *Self, v: T) !void {
            if (self.len == self.items.len) return error.Full;
            self.items[self.len] = v;
            self.len += 1;
        }
        pub fn pop(self: *Self) ?T {
            if (self.len == 0) return null;
            self.len -= 1;
            return self.items[self.len];
        }
    };
}

var s = try Stack(u32).init(a, 16);
defer s.deinit(a);
try s.push(42);
```

`Stack(u32)` is a function call that returns a *type*. The returned type is a fresh `struct` whose methods close over `T`. The compiler monomorphises the call — `Stack(u32)` and `Stack(u64)` produce distinct types with distinct code. This is exactly how C++ templates work under the hood, except the interface is just ordinary Zig instead of a separate template language.

`@This()` is a compile-time builtin that returns the innermost struct type currently being defined. It is how struct methods refer to "my own type" when writing generics — which is most of the time.

6.5 comptime blocks

Anywhere you can put a statement, you can put a `comptime` block, and the compiler runs it during compilation:

```
const PRIMES = comptime blk: {
    var arr: [25]u32 = undefined;
    var n: u32 = 2;
    var i: usize = 0;
    while (i < 25) : (n += 1) {
        var p = true;
        var d: u32 = 2;
        while (d * d <= n) : (d += 1) if (n % d == 0) { p = false; break; };
        if (p) { arr[i] = n; i += 1; }
    }
    break :blk arr;
};
```

`PRIMES` is a `[25]u32` whose bytes are computed during compilation and baked into the binary. There is no runtime cost. A compile-time block can allocate on a comptime allocator, build data structures, run arbitrary algorithms — anything you could write in plain Zig, within the usual comptime limits.

One caveat: the comptime evaluator is slower than the native CPU by about three orders of magnitude. If your comptime block is computing something huge, your build time suffers. Keep comptime work bounded and deliberate.

6.6 comptime if

If the condition to an `if`, `switch`, or `while` is known at compile time, the compiler evaluates the branch statically and elides the other one from the generated code. You can force this with a `comptime` keyword:

```
pub fn print(comptime fmt: []const u8, args: anytype) !void {
    inline for (std.meta.fields(@TypeOf(args))) |field| {
        // ... different codegen per field type ...
    }
}
```

`inline for` and `inline while` ask the compiler to unroll the loop — each iteration is emitted as its own statement, with the loop variable replaced by its concrete value. This is how format strings work in Zig's standard library: the format string is a comptime parameter, so the compiler expands it into straight-line code for each argument type.

6.7 What you can do with comptime (and what you can't)

At compile time, you can:

- Evaluate pure functions on comptime values.
- Construct and destructure types (`@typeInfo` , `@Type`).
- Read files via `@embedFile` .
- Generate struct types with `@Type` .
- Unroll loops with `inline for` .
- Dispatch on types with `switch` on `@TypeOf(x)` .

You cannot, at compile time, do anything that depends on a runtime value — dereference a pointer the compiler cannot trace, call a foreign (C) function, issue a syscall, or allocate through a runtime allocator. The compiler will explain what you tried and why it was impossible.

6.8 A worked example: a compile-time lookup table

Suppose we need fast character classification. CRCs, digit tests, CSS-quoting — all of them are "given this byte, return this byte" functions. The fastest version is a 256-entry table. The cleanest version is to *compute* the table at compile time from a predicate function:

```
const std = @import("std");

fn lookup(comptime classify: fn (u8) u8) [256]u8 {
    var table: [256]u8 = undefined;
    comptime var i: usize = 0;
    inline while (i < 256) : (i += 1) {
        table[i] = classify(@intCast(i));
    }
    return table;
}

fn upperize(c: u8) u8 {
    return if (c >= 'a' and c <= 'z') c - 32 else c;
}

const UPPER: [256]u8 = lookup(upperize);

pub fn shout(s: []u8) void {
    for (s) |*c| c.* = UPPER[c.*];
}
```

The function `lookup` runs at compile time; it builds a 256-entry table by invoking `classify` on every possible byte. `UPPER` is a constant baked into the binary. The runtime cost of `shout` is one table lookup and one store per character — faster than the branchier `if` inside the loop.

The striking thing is how little new syntax there is. `comptime`, `inline`, and the rule that "types are values" are the whole story. Once you see this one example, you see how the standard library's JSON parser, formatter, serialization helpers, and protocol decoders all use the same mechanism: they take a type as a comptime parameter and generate straight-line code for it.

6.9 Examples

Example A: a tagged union dispatcher at compile time

```
const Event = union(enum) {
    click: struct { x: i32, y: i32 },
    key:   u8,
    scroll: f32,
};

fn handle(e: Event) void {
    switch (e) {
        inline else => |payload, tag| {
            std.debug.print("{s}: {any}\n", .{ @tagName(tag), payload });
        },
    }
}
```

`inline else` generates one branch per variant; inside each branch, `payload` has the correctly-inferred type for that variant. One `switch` statement compiles into three specialised handlers, each typed.

Example B: a comptime-configured allocator wrapper

```
fn TrackingAllocator(comptime enabled: bool) type {
    return struct {
        child: std.mem.Allocator,
        total: usize = 0,

        const Self = @This();

        pub fn allocator(self: *Self) std.mem.Allocator {
            if (!enabled) return self.child;
            return .{ .ptr = self, .vtable = &vtable };
        }

        // ... vtable, etc. ...
    };
}
```

When `enabled` is `false`, the whole tracking machinery compiles to a no-op: the wrapper's `allocator()` returns the child directly, and the rest of the struct is dead code the compiler strips. This is the standard way to express "pay only for what you enable" in Zig — a single generic type, two behaviors chosen at compile time.

6.10 Exercises

1. Write a comptime function `matchesPattern(comptime pattern: []const u8, s: []const u8) bool` that does a tiny subset of glob matching (only `?` and `*`). Make `pattern` comptime. What changes inside the body compared to a pure-runtime version?

2. Build a compile-time multiplication table: a `[10][10]u32` whose entries are the products of their indices. How many bytes does it take up in the binary?

3. Write a generic `sum(comptime T: type, xs: []const T) T`. Constrain `T` so that the function only compiles for numeric types. Hint: `@typeInfo(T)`.

4. Read the source of `std.fmt.format`. What comptime magic makes `"{d}"` produce a different call than `"{s}"` at each call site?

We've only scratched the surface of comptime; Chapters 8 and 9 will push it much further. Before we get there, Chapter 7 revisits what comptime means when you are just trying to do one specific compile- time thing — and Chapter 10 turns to errors, which are the other half of Zig's unusual control-flow story.